A Software-Based Comparative Study of Error Sensitivity in Classical and Quantum Optimization Algorithms

Aayush Rajak

February 5, 2026

Classical computing processes information using bits that take definite values of 0 or 1, whereas quantum computing uses qubits that can exist in superpositions of states and become entangled, enabling fundamentally different computational behavior. [2] Quantum computing has the potential to improve the performance of certain computational tasks, particularly search and optimization problems. Many quantum algorithms promise theoretical speedups over classical methods; however, these results often rely on ideal assumptions that ignore noise, limited circuit depth, and other practical constraints of current quantum technology. As a result, there is a gap between theoretical expectations and the real-world behavior of quantum algorithms. This work aims to bridge that gap through a software-based comparison of classical and quantum optimization algorithms, focusing on their sensitivity to noise and errors using classical simulation. The primary goal of this research is to evaluate how quantum optimization algorithms perform under realistic execution conditions when compared to established classical heuristic methods. These problems are computationally challenging for the same underpinning reason that quantum computers can be powerful. While systems in classical physics exist in just one configuration, in quantum mechanics different configurations can coexist at the same time in a superposition of all of the possible states. [1] Instead of concentrating only on asymptotic complexity or idealized speedups, the study emphasizes practical performance, robustness, and solution quality in noisy environments. Optimization problems are chosen as the focus because they are central to computer science, widely used in practice, and commonly cited as candidates for near-term quantum advantage. This work compares three classical optimization approaches—simulated annealing, genetic algorithms, and greedy or local search heuristics—with three quantum techniques: Grover-based unstructured search, The Quantum Approximate Optimization Algorithm (QAOA) is a hybrid quantum classical variational algorithm designed to tackle combinatorial optimization problems., [3] and amplitude amplification. These algorithms are selected because they target similar problem classes, have well-understood theoretical foundations, and can be implemented using existing quantum simulation frameworks. To reflect current technological limitations, the experiments are restricted to small problem instances and low-depth quantum circuits. A key contribution is the design and implementation of a modular software framework for benchmarking classical and quantum algorithms under controlled conditions. The framework supports configurable noise models for quantum

circuits, repeated probabilistic simulations, and systematic parameter exploration. Performance is evaluated using metrics such as solution quality, stability across runs, circuit depth, gate counts, qubit requirements, runtime overhead, and sensitivity to increasing noise levels. This standardized approach enables fair, reproducible, and transparent comparisons between classical and quantum methods. Using this framework, experiments are conducted on representative optimization problems. The results show that although quantum algorithms may offer theoretical advantages, their performance often degrades rapidly when noise and realistic constraints are introduced. In many scenarios, classical heuristic algorithms demonstrate greater robustness and produce higher-quality solutions. The analysis also identifies specific components of quantum algorithms that are particularly sensitive to error, including oracle construction in Grover-based methods and parameter optimization in QAOA. The work further discusses why theoretical quantum advantages are difficult to realize in practice. Factors such as error accumulation, limited circuit depth, and classical preprocessing overhead are shown to significantly affect performance. Rather than dismissing the potential of quantum optimization, this study highlights the importance of hybrid classical–quantum approaches and realistic evaluation methodologies. Overall, this work provides a practical, software-focused assessment of classical and quantum optimization algorithms under realistic conditions. By emphasizing error sensitivity and empirical behavior, it contributes to a clearer and more balanced understanding of the capabilities and limitations of near-term quantum computing.

Figure 1: Archie

Appendix

- Set up project environment and libraries. (Create Python environment, install Qiskit, NumPy, SciPy, Matplotlib, and folder structure.)
- Generate optimization problem instances. (Create graphs, Max-Cut, or small TSP problems with configurable size and randomness.)
- Implement simulated annealing. (Classical optimization algorithm for comparison with quantum methods, log runtime and solution quality.)
- Implement genetic algorithm. (Classical evolutionary approach with mutation, crossover, and selection, log metrics.)
- Implement greedy/local search algorithm. (Simple heuristic for baseline performance measurement.)
- Implement Grover-based quantum search. (Simulate unstructured search with configurable qubits and output best solution.)
- Implement QAOA quantum algorithm. (Hybrid quantum-classical variational optimization with parameter updates and solution logging.)
- Implement amplitude amplification. (Quantum technique for sampling and probability improvement with small problem instances.)
- Add noise models to quantum simulations. (Include depolarizing, dephasing, and measurement noise; allow toggling noise on/off.)
- Create experiment runner. (Run multiple algorithms on multiple problem instances and collect metrics like solution quality, runtime, and stability.)
- Store experiment results. (Save outputs and metadata in CSV/JSON for reproducibility and analysis.)
- Visualize results. (Plot solution quality, runtime, and convergence for classical vs quantum methods.)

References

[1] Daley, A. J., Bloch, I., Kokail, C., Flannigan, S., Pearson, N., Troyer, M., and Zoller, P. Practical quantum advantage in quantum simulation.

[2] Wong, T. G. *Introduction to Classical and Quantum Computing*. Self-published, 2019.

[3] Zhou, L., Wang, S.-T., Choi, S., Pichler, H., and Lukin, M. D. Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices.